
CONFIDENTIAL HANDOVER

XYZ London Project Handover

Architecture, operations, and developer notes for the MedusaJS store, Next.js storefront, and Sanity CMS.

PREPARED FOR

Client and incoming developer

STACK

Medusa 2.15 · Next.js 15 · Sanity 3

DATE

2 September 2026

SOURCE

HANDOVER.md in the project repository

This document is the handover for **XYZ London**: a luxury streetwear / digital-fashion store built on MedusaJS, Next.js, and Sanity. It is written for two audiences.

Audience	Start here
Client / brand / merchandiser	What you received , Who edits what , Day-to-day operations
Incoming developer	Architecture , Local development , Environment variables , How the storefront is built

Read this document before changing production. Official product docs are linked at the end; this file describes **this** codebase, not generic Medusa or Next.js.

Table of contents

1. [What you received](#)
 2. [Brand information architecture](#)
 3. [Architecture](#)
 4. [Who edits what](#)
 5. [Repository layout](#)
 6. [Stack and versions](#)
 7. [Access and accounts to transfer](#)
 8. [Local development](#)
 9. [Environment variables](#)
 10. [Production / Railway](#)
 11. [Day-to-day operations](#)
 12. [Payments](#)
 13. [How the storefront is built](#)
 14. [Virtual try-on](#)
 15. [Digital Form commerce](#)
 16. [Email](#)
 17. [Search](#)
 18. [File storage \(MinIO\)](#)
 19. [SEO](#)
 20. [Custom backend APIs and admin widgets](#)
 21. [Catalog scripts](#)
 22. [Testing](#)
 23. [Known issues and follow-up work](#)
 24. [Security notes](#)
 25. [Common tasks](#)
 26. [Official documentation](#)
 27. [Glossary](#)
-

1. What you received

XYZ London is a **headless commerce** site. The customer-facing website, the commerce engine, and the marketing CMS are three separate applications that talk to each other.

App	What it is	Typical URL (local)	Who uses it
Storefront	Next.js 15 website customers see	http://localhost:8000	Shoppers
Backend (Medusa)	Commerce API + Admin dashboard	http://localhost:9000 — admin at /app	Merchandisers, ops, developers
Sanity Studio	Marketing / editorial CMS	http://localhost:3333	Brand / content editors

The repo is a monorepo based on the [Medusa 2.0 Railway boilerplate](#), heavily customized for XYZ London (Physical Form, Digital Form, OOO, Journal, virtual try-on, X/Y/Z lines).

Git remote (current): <https://github.com/maleekcodes/medusajs-2.0-for-railway-boilerplate.git>

Transfer this repository (or a fork) into the client's GitHub organisation as part of handover.

What is live vs what is content

- **Physical products** (tees, caps, inventory, prices, variants, checkout, orders) live in **Medusa**.
- **Homepage copy, About, Journal, Digital Form merchandising, OOO page, footer, most SEO** live in **Sanity**.
- **Legal page body copy** (privacy, terms, shipping) is **hardcoded** in the storefront. Only the SEO titles/descriptions for those pages are in Sanity.
- **Contact page body** is **hardcoded** (`contact@wearxyz.co`).

2. Brand information architecture

The storefront is organised around three brand pillars plus supporting pages.

<code>/ {country} /</code>	Homepage
<code>/ {country} / store</code>	Physical Form (Medusa catalog)
<code>/ {country} / digital</code>	Digital Form (Sanity + Stripe Checkout)
<code>/ {country} / digital / {slug}</code>	Digital product
<code>/ {country} / digital / {slug} / success</code>	Digital purchase success + download
<code>/ {country} / private-expressions</code>	000 / Highest Expression
<code>/ {country} / about</code>	About
<code>/ {country} / journal</code>	Journal (Editorial + Lookbook)
<code>/ {country} / journal / {slug}</code>	Journal post
<code>/ {country} / virtual-try-on</code>	Try-on explainer
<code>/ {country} / ar-fit</code>	Permanent redirect → <code>/ virtual-try-on</code>
<code>/ {country} / contact</code>	Contact
<code>/ {country} / products / {handle}</code>	Physical product (Medusa)
<code>/ {country} / categories / {handle}</code>	Category (Tees, Caps, ...)
<code>/ {country} / collections / {handle}</code>	Collection (including X / Y / Z)
<code>/ {country} / cart</code>	Cart (physical)
<code>/ {country} / checkout</code>	Checkout (physical, Medusa + Stripe)
<code>/ {country} / search</code>	Search overlay
<code>/ {country} / results / {query}</code>	Search results
<code>/ {country} / account</code>	Account / login
<code>/ {country} / order / confirmed / {id}</code>	Physical order confirmation
<code>/ {country} / content / privacy-policy</code>	
<code>/ {country} / content / terms-of-use</code>	
<code>/ {country} / content / shipping-policy</code>	

Every customer URL is prefixed with a **country code** (for example `/ gb / store`). Middleware adds it automatically. `{country}` comes from Medusa **regions**.

Navigation

Desktop left: Physical Form · Digital Form · OOO

Desktop centre: XYZ London mark → home

Desktop right: About · Journal · Try-on · Search (if enabled) · Account · Cart

On OOO routes the chrome uses a light theme (`#EBEBEB`). Digital routes use a near-black theme.

Physical Form merchandising model (important)

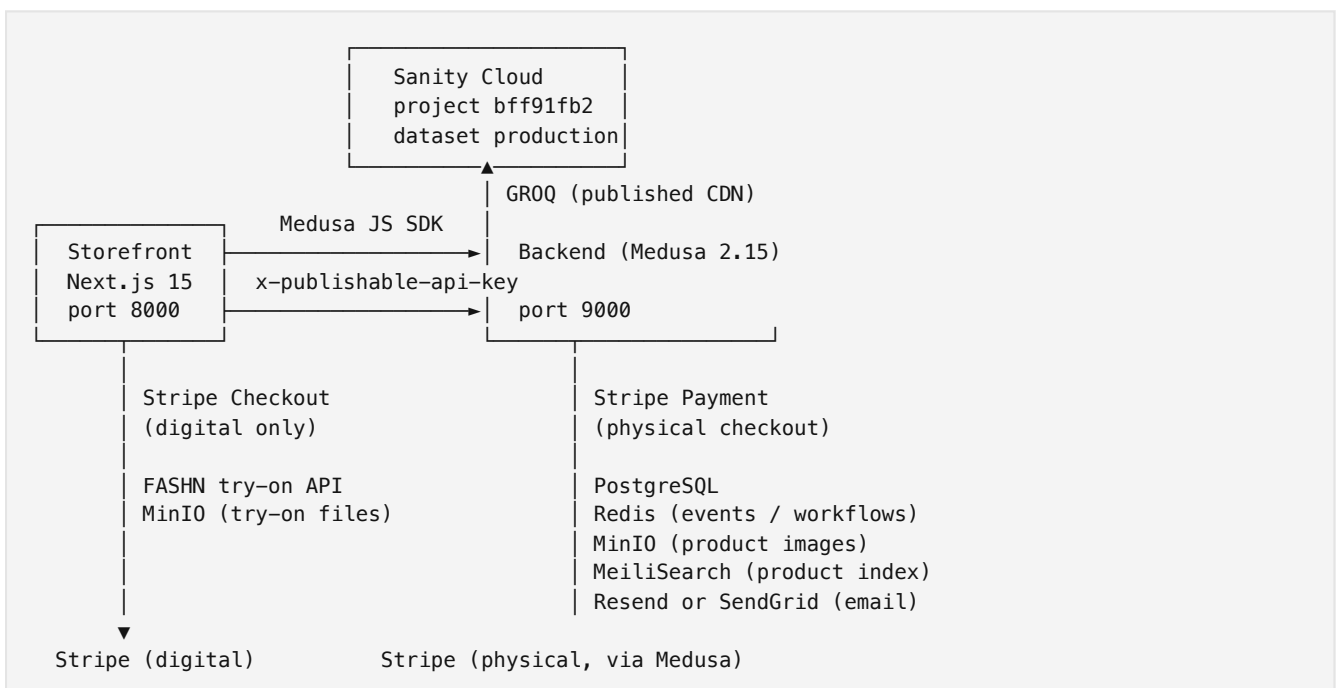
Do **not** treat X, Y, and Z as product categories.

Concept	Medusa object	Handles	Storefront use
Line	Collection	<code>x</code> , <code>y</code> , <code>z</code>	Groups products into X-Line / Y-Line / Z-Line
Type	Product category	<code>tees</code> , <code>caps</code> , ...	Physical Form tabs and <code>/categories/{handle}</code>
Future Forms	Categories with <code>metadata.coming_soon = true</code>	e.g. hoodies	Coming-soon grid on Physical Form

A product should typically have:

1. One **type category** (Tees or Caps, etc.)
2. One **line collection** (X, Y, or Z)
3. Rich **metadata** (tagline, composition, GSM, size guide, ...) edited via Admin widgets

3. Architecture



Request flow — physical purchase

1. Storefront reads products/regions from Medusa Store API.
2. Customer adds to cart (`_medusa_cart_id` cookie).
3. Checkout at `/country/checkout` creates Medusa payment sessions.

4. Stripe.js (publishable key) collects the card; Medusa backend (`STRIPE_API_KEY`) charges.
5. `order.placed` subscriber sends the order-confirmation email.

Request flow — digital purchase

1. Digital catalog comes from Sanity (`digitalFormPage.digitalProducts[]`).
2. Customer runs virtual try-on (FASHN). Output is stored in MinIO under `digital-tryon/` .
3. Storefront `POST /api/stripe/digital-checkout` creates a **Stripe Checkout Session** with the storefront's `STRIPE_SECRET_KEY` .
4. After payment, `/{{country}}/digital/{slug}/success` + `/api/digital/download?session_id=` serve a signed MinIO URL for the try-on file.

Physical and digital payments are **two different Stripe integrations**. They can share the same Stripe account, but they use different keys and different webhook endpoints.

Data ownership

Data	System of record
SKUs, variants, prices, stock, regions, tax, shipping, customers, physical orders	Medusa / PostgreSQL
Product photography for physical goods	MinIO (via Medusa File module)
Homepage, About, Journal, Digital merchandising, OOO, footer, global SEO	Sanity
Legal / contact page body	Storefront source code
Digital try-on outputs	MinIO (<code>digital-tryon/</code> , <code>physical-tryon/</code>)
Product search index	MeiliSearch

4. Who edits what

Client / merchandiser — no code required

Task	Tool
Add / edit physical products, prices, variants, images	Medusa Admin → Products
Storefront copy (tagline, composition, size guide, ...)	Product page widgets in Admin
Assign X / Y / Z line	Product page “collection” widget
Assign Tees / Caps / create type categories	Product page “categories” widget
Mark a category Coming Soon / hide it / set Future Forms shape	Category page widget
Inventory, regions, shipping, tax, discounts	Medusa Admin
Orders (physical)	Medusa Admin → Orders
Homepage, About, Journal, Digital Form, OOO, footer, SEO	Sanity Studio
Invite another admin user	Medusa Admin → Settings → Users

Developer — code or env required

Task	Where
Change legal or contact page wording	storefront/src/modules/legal/ and contact/
Change nav structure	storefront/src/modules/layout/
New storefront page or component	storefront/src/app/ + storefront/src/modules/
New Medusa module, subscriber, or Admin widget	backend/src/
New Sanity document type	sanity/schemas/ + storefront GROQ in storefront/src/lib/sanity/queries.ts
Env / Stripe / CORS / Railway	Hosting dashboards + env files
Seed / wipe / script catalog	backend/src/scripts/

5. Repository layout

/	
├─ HANDOVER.md	← this document
├─ README.md	← original Railway boilerplate readme
├─ CLAUDE.md	← notes for coding agents
├─ backend/	← MedusaJS server + Admin
│ └─ medusa-config.js	← modules gated by env vars
│ └─ src/	
│ └─ api/	← custom HTTP routes
│ └─ admin/widgets/	← XYZ Admin widgets
│ └─ lib/constants.ts	← env loading + validation
│ └─ modules/	
│ └─ minio-file/	← S3-compatible storage provider
│ └─ email-notifications/	← Resend + React Email
│ └─ subscribers/	← order-placed, invite-created
│ └─ scripts/	← seed + XYZ catalog scripts + assets/
│ └─ .nvmrc	← Node 22.11.0
├─ storefront/	← Next.js 15 storefront
│ └─ src/	
│ └─ app/	← App Router pages + API routes
│ └─ lib/	← Medusa SDK, Sanity, SEO, digital/try-on
│ └─ modules/	← UI grouped by feature
│ └─ middleware.ts	← country-code enforcement
│ └─ styles/globals.css	
│ └─ e2e/	← Playwright tests
├─ sanity/	← Sanity Studio
│ └─ schemas/	← 9 document types
│ └─ seed.ts	← initial CMS content
│ └─ sanity.config.ts	

There is **no root package.json** . Install and run each app from its own folder.

6. Stack and versions

Layer	Version / choice
Node.js	22.x (backend .nvmrc = v22.11.0)

Layer	Version / choice
Package manager	pnpm 9.10.0
Medusa	2.15.2 (root README still says 2.12.1 — ignore that)
Next.js	15.5
React	19.0.4 (storefront) / 18 (backend admin, Sanity)
Sanity	3.30
Stripe (backend physical)	@medusajs/payment-stripe 2.15.2
Stripe (storefront digital)	stripe ^22 + Checkout Sessions
Search	MeiliSearch via @rokmohar/medusa-plugin-meilisearch 1.3.5
Email	Resend 4.0.1 (or SendGrid) + React Email
Storage	MinIO (S3 API)
Try-on provider	FASHN (VIRTUAL_TRYON_API_KEY)
Hosting (intended)	Railway (Postgres, Redis, MinIO, MeiliSearch, backend, storefront)

7. Access and accounts to transfer

Hand these to the client (or confirm they already own them). **Do not put secrets in this repo.**

Service	What to transfer	Notes
GitHub	Repo ownership or a new remote	Current: maleekcodes/medusajs-2.0-for-railway-boilerplate
Railway	Project + all services + env vars	Backend, storefront, Postgres, Redis, MinIO, MeiliSearch
Medusa Admin	Admin email / password	Create a client user; rotate the old password
Sanity	Project bff91fb2	sanity.io/manage — add the client as Admin/Owner
Stripe	Account + API keys + webhooks	Same account for physical + digital, or split if desired
Resend (or SendGrid)	API key + sending domain	Order + invite emails
MinIO	Endpoint, access key, secret, bucket medusa-media	Product images + try-on files
MeiliSearch	Host + admin key + search-only key	Storefront uses the search key
FASHN	VIRTUAL_TRYON_API_KEY	Digital + physical try-on
Domain / DNS	Storefront + backend + admin	Point at Railway (or new host)
HubSpot (OOO form)	Form embed URL	Stored on Sanity privateExpressionsPage.hubspotFormUrl
Google Search Console / Analytics	If already set up	Not configured in this repo

8. Local development

Prerequisites

- Node.js 22.x (`nvm use` in `backend/` reads `.nvmrc`)
- pnpm 9.10.0 (`corepack enable` then `corepack prepare pnpm@9.10.0 --activate`)
- PostgreSQL (local or Railway `DATABASE_URL`)
- Optional locally: Redis, MinIO, MeiliSearch. Without them the backend falls back (in-memory events, local file storage, no search plugin).

Backend

```
cd backend
pnpm install
# Create backend/.env (there is no committed .env.template – copy the table in §9)
pnpm ib          # first time only: migrate + seed if the DB is empty
pnpm dev         # http://localhost:9000 – Admin: http://localhost:9000/app
```

If the database is empty, `pnpm ib` (`init-backend` from `medusajs-launch-utils`) runs migrations, `db:sync-links`, `pnpm seed`, and optionally creates an admin user from `MEDUSA_ADMIN_EMAIL` / `MEDUSA_ADMIN_PASSWORD`.

Then create XYZ collections and catalog (only if they are not already in this database):

```
pnpm exec medusa exec ./src/scripts/ensure-xyz-collections.ts
# then the create-* product scripts listed in §21
pnpm exec medusa exec ./src/scripts/sync-storefront-copy.ts
```

Admin login: after seed, use the email/password you set, or create a user:

```
cd backend
npx medusa user -e you@client.com -p 'a-strong-password'
```

Reset an existing admin password:

```
MEDUSA_ADMIN_EMAIL=you@client.com MEDUSA_ADMIN_PASSWORD='new-password' \
pnpm exec medusa exec ./src/scripts/reset-admin-password.ts
```

Publishable API key: Admin → Settings → Publishable API Keys. Seed creates a key titled **Webshop**. Copy it into the storefront as `NEXT_PUBLIC_MEDUSA_PUBLISHABLE_KEY`.

After changing env vars, delete `backend/.medusa/server/` — Medusa caches compiled config there.

Storefront

```
cd storefront
pnpm install
cp .env.example .env.local
# fill NEXT_PUBLIC_MEDUSA_PUBLISHABLE_KEY and Sanity IDs at minimum
pnpm dev          # waits for backend, then http://localhost:8000
```

`pnpm dev` / `pnpm build` call `await-backend` then `launch-storefront` (`medusajs-launch-utils`). The backend must be reachable on `NEXT_PUBLIC_MEDUSA_BACKEND_URL` (default `http://localhost:9000`).

Build will **fail** if `NEXT_PUBLIC_MEDUSA_PUBLISHABLE_KEY` is missing (`check-env-variables.js`).

Sanity Studio

```
cd sanity
pnpm install
cp .env.example .env
# SANITY_STUDIO_PROJECT_ID=bff91fb2
# SANITY_STUDIO_DATASET=production
pnpm dev      # http://localhost:3333
```

You must be logged into the Sanity CLI / browser with access to project `bff91fb2`.

Seed CMS **only** on a new empty dataset (create-if-missing; will not overwrite existing documents):

```
SANITY_TOKEN=<editor-write-token> pnpm seed
```

Deploy a hosted Studio:

```
pnpm deploy      # sanity deploy - requires `sanity login`
```

Typical local ports

Service	Port
Storefront	8000
Backend + Admin	9000
Sanity Studio	3333
React Email preview	3002 (cd backend && pnpm email:dev)
MeiliSearch (if local)	7700

9. Environment variables

Never commit real `.env` / `.env.local` files.

9.1 Backend (backend/ .env)

There is **no** `.env.template` in the repo. Create `.env` from this table.

Required (process exits without these)

Variable	Purpose
<code>DATABASE_URL</code>	PostgreSQL connection string
<code>JWT_SECRET</code>	Signs admin / customer JWTs
<code>COOKIE_SECRET</code>	Signs cookies

Core

Variable	Default	Purpose
<code>NODE_ENV</code>	<code>development</code>	Environment
<code>PORT</code>	<code>9000</code>	HTTP port
<code>BACKEND_PUBLIC_URL</code>	—	Public backend URL (preferred over Railway fallback)

Variable	Default	Purpose
RAILWAY_PUBLIC_DOMAIN_VALUE	—	Railway-injected public domain
REDIS_URL	unset	Enables Redis event bus + workflow engine
ADMIN_CORS	unset	Admin UI allowed origins
AUTH_CORS	unset	Auth endpoint allowed origins
STORE_CORS	unset	Storefront allowed origins — set this in production
MEDUSA_WORKER_MODE	shared	shared server worker . worker skips seed on start
MEDUSA_DISABLE_ADMIN	false	true hides /app
MEDUSA_ADMIN_EMAIL	—	Created on first init-backend seed
MEDUSA_ADMIN_PASSWORD	—	Created on first seed; also used by reset script

CORS values are comma-separated origins, e.g. `https://wearxyz.co,https://www.wearxyz.co`.

MinIO (all three required to enable cloud storage; otherwise local static/)

Variable	Purpose
MINIO_ENDPOINT	Host, with or without <code>https://</code>
MINIO_ACCESS_KEY	Access key
MINIO_SECRET_KEY	Secret key
MINIO_BUCKET	Default <code>medusa-media</code>

Email — configure either Resend or SendGrid

Variable	Purpose
RESEND_API_KEY	Resend
RESEND_FROM_EMAIL or RESEND_FROM	From address
SENDGRID_API_KEY	SendGrid
SENDGRID_FROM_EMAIL or SENDGRID_FROM	From address

If neither pair is complete, the notification module is not loaded and order emails will not send.

Stripe (physical checkout via Medusa)

Variable	Purpose
STRIPE_API_KEY	Secret key — enables <code>@medusajs/payment-stripe</code>
STRIPE_WEBHOOK_SECRET	Loaded in <code>constants.ts</code> but commented out in <code>medusa-config.js</code> — not wired today

MeiliSearch

Variable	Purpose
MEILISEARCH_HOST	MeiliSearch URL
MEILISEARCH_ADMIN_KEY	Admin key for indexing
MEILISEARCH_MASTER_KEY	Used by <code>init-backend</code> on Railway to auto-fetch the admin key

9.2 Storefront (storefront/.env.local)

Template: storefront/.env.example .

Required

Variable	Purpose
NEXT_PUBLIC_MEDUSA_PUBLISHABLE_KEY	Medusa Store API key (build fails without it)

Medusa / site

Variable	Default	Purpose
NEXT_PUBLIC_MEDUSA_BACKEND_URL	http://localhost:9000	Backend API
NEXT_PUBLIC_BASE_URL	http://localhost:8000	Canonical storefront URL (SEO, sitemap, Stripe redirects)
NEXT_PUBLIC_DEFAULT_REGION	us	Fallback country code. Seeded region uses Europe (gb , de , ...). Set this to a country that exists in Medusa (often gb)
NEXT_PUBLIC_MINIO_ENDPOINT	—	Allowed image host in next.config.js
NEXT_PUBLIC_FEATURE_SEARCH_ENABLED	unset	Truthy → show Search in nav
NEXT_PUBLIC_SEARCH_ENDPOINT	http://127.0.0.1:7700	MeiliSearch
NEXT_PUBLIC_SEARCH_API_KEY	test_key	Search-only key (not the admin key)
NEXT_PUBLIC_INDEX_NAME	products	MeiliSearch index
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	—	Stripe.js on physical checkout (pk_...)
NEXT_PUBLIC_STRIPE_KEY	—	Alias for the publishable key
NEXT_PUBLIC_PAYPAL_CLIENT_ID	—	PayPal (provider must also be on the backend)
NEXT_PUBLIC_PRIVATE_EXPRESSIONS_TYPEFORM_URL	—	Override OOO embed URL
PORT	launcher / 8000 locally	Host port

Sanity (must match Studio)

Variable	Purpose
SANITY_PROJECT_ID	Same as SANITY_STUDIO_PROJECT_ID (default project bff91fb2)
SANITY_DATASET	Usually production
SANITY_API_VERSION	2024-01-01
SANITY_TOKEN	Optional. Leave unset to read published content via CDN
NEXT_PUBLIC_SANITY_*	Accepted as aliases

If `SANITY_PROJECT_ID` is empty, the storefront uses hardcoded fallbacks (footer, OOO, etc.).

Digital / try-on (server-only — never `NEXT_PUBLIC_`)

Variable	Purpose
<code>VIRTUAL_TRYON_API_KEY</code>	FASHN API key. Also accepts legacy <code>FASHN_API_KEY</code>
<code>VIRTUAL_TRYON_PROVIDER_BASE_URL</code>	Default <code>https://api.fashn.ai/v1</code>
<code>STRIPE_SECRET_KEY</code>	Stripe Checkout for digital products
<code>STRIPE_WEBHOOK_SECRET</code>	Verifies <code>POST /api/stripe/webhook</code>
<code>DIGITAL_PREVIEW_HMAC_SECRET</code>	Signs <code>/api/try-on/preview</code> URLs. Falls back to <code>STRIPE_WEBHOOK_SECRET</code> or <code>MINIO_SECRET_KEY</code>
<code>MINIO_ENDPOINT</code> / <code>MINIO_ACCESS_KEY</code> / <code>MINIO_SECRET_KEY</code>	Persist try-on files
<code>MINIO_BUCKET</code>	Default <code>medusa-media</code>
<code>MINIO_USE_SSL</code>	Default <code>true</code>
<code>MINIO_SERVER_URL</code>	Full URL if endpoint is host-only
<code>MINIO_REGION</code>	Default <code>us-east-1</code>

9.3 Sanity Studio (`sanity/.env`)

```
SANITY_STUDIO_PROJECT_ID=bff91fb2
SANITY_STUDIO_DATASET=production
SANITY_TOKEN=                # only for pnpm seed / migrate
```

10. Production / Railway

The original template is designed for Railway. This repo has **no** `Dockerfile`, `railway.toml`, or `nixpacks.toml`. Railway Nixpacks detects Node and uses each service's `package.json` scripts.

Suggested Railway services

Service	Root directory	Start command	Notes
Backend	<code>backend</code>	<code>pnpm start</code>	Runs <code>init-backend</code> then <code>.medusa/server medusa start</code>
Storefront	<code>storefront</code>	<code>pnpm start</code>	Set <code>PORT</code> as Railway injects
Postgres	<code>plugin</code>	—	<code>DATABASE_URL</code>
Redis	<code>plugin</code>	—	<code>REDIS_URL</code>
MinIO	<code>template / image</code>	—	Public bucket <code>medusa-media</code>
MeiliSearch	<code>template / image</code>	—	Host + keys

Backend production commands

```
pnpm build
# medusa build + src/scripts/postBuild.js
# copies lockfile + .env into .medusa/server and runs prod pnpm i

pnpm start
# init-backend (migrate/seed if empty, Meili admin key fetch)
# then: cd .medusa/server && medusa start --verbose
```

Storefront production

```
pnpm build      # waits for backend, then next build
pnpm start      # launch-storefront start
```

`next.config.js` sets `eslint.ignoreDuringBuilds` and `typescript.ignoreBuildErrors` to `true`. Type errors will not fail a production build — fix them in CI if you add a stricter pipeline.

Production env checklist

Backend

- ☐ `DATABASE_URL`, `JWT_SECRET`, `COOKIE_SECRET` (long random strings)
- ☐ `BACKEND_PUBLIC_URL` = public backend URL (https)
- ☐ `STORE_CORS` = storefront origin(s)
- ☐ `ADMIN_CORS` / `AUTH_CORS` = admin origin(s)
- ☐ `REDIS_URL`
- ☐ MinIO trio + bucket
- ☐ `STRIPE_API_KEY` (`sk_live_...` when going live)
- ☐ Resend or SendGrid pair; verify sending domain
- ☐ MeiliSearch host + admin key

Storefront

- ☐ `NEXT_PUBLIC_MEDUSA_BACKEND_URL` = public backend
- ☐ `NEXT_PUBLIC_BASE_URL` = public storefront (no trailing path)
- ☐ `NEXT_PUBLIC_MEDUSA_PUBLISHABLE_KEY` = Webshop key
- ☐ `NEXT_PUBLIC_DEFAULT_REGION` = a real country in your region (e.g. `gb`)
- ☐ `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` matching the backend Stripe account
- ☐ Sanity project + dataset
- ☐ `NEXT_PUBLIC_MINIO_ENDPOINT` so Next/image allows product URLs
- ☐ Digital: `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, MinIO, `VIRTUAL_TRYON_API_KEY`

Stripe webhooks to register

Endpoint	Used for
<code>{STOREFRONT}/api/stripe/webhook</code>	Digital Checkout (storefront verifies signature)
<code>{BACKEND}/hooks/payment/stripe_stripe</code>	Medusa physical Stripe (standard Medusa path). <code>webhookSecret</code> is currently commented out in <code>medusa-config.js</code> — enable it before relying on webhook-driven payment status

After deploy

1. Open `{BACKEND}/app` and confirm Admin login.
2. Confirm publishable key + default sales channel.
3. Place a **test** physical order (Stripe test mode) and a **test** digital checkout if digital is in use.
4. Confirm product images load (MinIO public-read).
5. Publish all Sanity singletons (drafts are invisible to the storefront CDN).
6. Set Search Console / sitemap: `{STOREFRONT}/sitemap.xml`, `{STOREFRONT}/robots.txt`.

Health probes:

- Storefront: `GET /api/healthcheck`
- Backend smoke: `GET /store/custom` (200 empty body)

11. Day-to-day operations

11.1 Medusa Admin

URL: `{BACKEND_URL}/app`

Typical sections:

Admin area	Use
Products	Physical catalog, variants, images, metadata widgets
Collections	X / Y / Z lines (handles must stay <code>x</code> , <code>y</code> , <code>z</code>)
Categories	Tees, Caps, Future Forms
Orders	Physical orders, fulfillment
Customers	Store accounts
Inventory	Stock at “European Warehouse” (seed default)
Settings → Regions	Countries, currency, payment providers
Settings → Tax	Tax regions
Settings → Locations & shipping	Stock location + shipping options
Settings → Publishable API Keys	Storefront key (“Webshop”)
Settings → Users / Invites	Admin access (invite email uses Resend)

Admin file upload limit is **25 MB** (`medusa-config.js`).

11.2 Add a physical product (recommended path)

1. **Admin** → **Products** → **Create**.
2. Title can be `X _ Money Orders Tee` — the storefront strips the `X _` prefix unless `display_title` is set.
3. Add options: **Color** (or Colour / Finish) and **Size** as needed.
4. Upload images. First / thumbnail is used in grids and try-on.
5. On the product page, use the XYZ widgets:
 - **Storefront copy** — tagline, overview, composition, GSM, fit, care, shipping, size guide JSON.
 - **Swatches** — map each color value to a hex in `metadata.swatch_colors`.
 - **Collection** — assign **X**, **Y**, or **Z** (sets `metadata.collection_line`).

- **Categories** — assign **Tees** or **Caps** (not X/Y/Z).

6. Publish the product and attach it to the **Default Sales Channel**.
7. Set inventory at the stock location.
8. Set prices for the region currency.

Size guide JSON example:

```
{"columns":["Fit","Notes"],"rows":[["S","Chest 50cm"],["M","Chest 53cm"]]}
```

Color-specific taglines: metadata key `tagline_green` (or `tagline_<normalized-color>`), or a `taglines` JSON map.

11.3 Categories and Future Forms

On **Admin** → **Categories** → **[category]**, the storefront settings widget writes:

Metadata	Effect
<code>coming_soon</code>	Shown in the Physical Form “Future Forms” / Coming Soon block
<code>hide_on_store</code>	Hidden from Physical Form tabs
<code>shape</code>	Geometry used in Future Forms teasers (<code>hexagon</code> , <code>square</code> , <code>rhombus</code> , ...)
<code>subtitle</code>	Optional subtitle

Active type categories (not coming soon, not hidden, not X/Y/Z) become **tabs** on `/store` .

11.4 Sanity Studio — content editors

Studio title: **XYZ London CMS**. Default project `bff91fb2` , dataset `production` .

There is **no custom desk / singleton UI**. Documents appear as normal Sanity types. Keep **one document** of each singleton type. Seeded IDs:

Type	Expected <code>id</code>	Storefront
<code>siteSettings</code>	<code>siteSettings</code>	Global SEO, org JSON-LD, social
<code>siteFooter</code>	<code>siteFooter</code>	Footer (merged with hardcoded defaults)
<code>homePage</code>	<code>homePage</code>	<code>/</code>
<code>aboutPage</code>	<code>aboutPage</code>	<code>/about</code>
<code>arFitPage</code>	<code>arFitPage</code>	<code>/virtual-try-on</code>
<code>digitalFormPage</code>	<code>digitalFormPage</code>	<code>/digital</code> and <code>/digital/[slug]</code>
<code>privateExpressionsPage</code>	<code>privateExpressionsPage</code>	<code>/private-expressions</code> + homepage OOO teaser
<code>staticPageSeo</code>	<code>staticPageSeo</code>	SEO only for store, journal, contact, legal, cart, checkout, search, 404
<code>journalPost</code>	one per article	<code>/journal</code> , <code>/journal/[slug]</code>

Journal categories

- Editorial index: `Theory` , `Process` , `Dialogue`
- Lookbook / campaign index: `Lookbook` , `Campaign`

Digital products are rows **inside** `digitalFormPage` , not separate documents. After adding images, **Publish** the page — the storefront only reads published CDN content. There is no draft preview.

OOO form: `hubspotFormUrl` on `privateExpressionsPage` . Optional override:
`NEXT_PUBLIC_PRIVATE_EXPRESSIONS_TYPEFORM_URL` .

Publishing: Edit → Publish → refresh the storefront. CDN cache is usually seconds, not minutes.

Do not create a second Home Page / About Page. GROQ uses `*[_type == "homePage"] [0]` — a duplicate would be undefined which document wins.

11.5 Legal and contact

Page	Body copy	SEO
Privacy, Terms, Shipping	Hardcoded React templates in <code>storefront/src/modules/legal/templates/</code>	Sanity <code>staticPageSeo</code>
Contact	Hardcoded <code>storefront/src/modules/contact/templates/contact-template.tsx</code>	Sanity <code>staticPageSeo.contact</code>

Contact email in templates and fallbacks: contact@wearxyz.co.

12. Payments

Physical (Medusa cart checkout)

- Provider: `@medusajs/payment-stripe` when `STRIPE_API_KEY` is set.
- Storefront mounts Stripe.js with `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` (or `NEXT_PUBLIC_STRIPE_KEY`).
- **The publishable key and secret key must belong to the same Stripe account.**
- PayPal is wired in the UI (`pp_paypal_paypal`) if `NEXT_PUBLIC_PAYPAL_CLIENT_ID` is set **and** a PayPal provider exists on the backend (not configured in `medusa-config.js` today).
- Seeded region payment provider is `pp_system_default` (manual / test). After adding Stripe, enable the Stripe provider on the **Europe** (or live) region in Admin.

Digital (Stripe Checkout, bypasses Medusa cart)

- `STRIPE_SECRET_KEY` on the **storefront**.
- Session metadata stores `slug` , `prediction_id` , MinIO key.
- Currencies allowed in code: **gbp, usd, eur**.
- Checkout is blocked until a try-on file exists in MinIO.
- Download is gated on `session.payment_status === "paid"` .

Going live with Stripe

1. Switch keys from `sk_test_` / `pk_test_` to live.
 2. Register both webhook URLs (\$10).
 3. Uncomment and set `webhookSecret` in `backend/medusa-config.js` for physical payments.
 4. Place a real small order in production, then refund.
-

13. How the storefront is built

Routing

- App Router under `storefront/src/app/[countryCode]/`.
- `(main)` = nav + footer. `(checkout)` = minimal chrome.
- `storefront/src/middleware.ts`:
 - Loads Medusa regions (1 hour cache).
 - Redirects any path without a valid country prefix.
 - Country resolution: URL → Vercel `x-vercel-ip-country` → `NEXT_PUBLIC_DEFAULT_REGION` → first region.
 - Sets `_medusa_cart_id` when `?cart_id=` is present.

Data fetching

Concern	Location
Medusa SDK client	<code>storefront/src/lib/config.ts</code>
Cart, products, customer, regions, orders	<code>storefront/src/lib/data/*.ts</code>
Server Actions (mutations)	<code>storefront/src/modules/*/actions.ts</code>
Sanity GROQ	<code>storefront/src/lib/sanity/queries.ts</code>
SEO helpers	<code>storefront/src/lib/seo/</code>
Digital / try-on / MinIO	<code>storefront/src/lib/digital/</code>

Feature modules (`storefront/src/modules/`)

Folder	Purpose
<code>home/components/xyz/</code>	Homepage sections
<code>store/</code>	Physical Form catalog
<code>products/</code>	Physical PDP
<code>digital/</code>	Digital Form index
<code>digital-product/</code>	Digital PDP + Stripe
<code>private-expressions/</code>	OOO
<code>journal/</code>	Editorial + lookbook
<code>virtual-try-on/</code>	Try-on marketing page
<code>about/</code> , <code>contact/</code> , <code>legal/</code>	Static-ish pages
<code>layout/</code>	Nav, footer
<code>checkout/</code> , <code>cart/</code> , <code>account/</code>	Commerce
<code>common/components/xyz/</code>	Logo, Container, ClientOnly

Design tokens (storefront/tailwind.config.js)

Token	Hex	Use
deepBlack	#0F0F0F	Body / digital
concrete	#F4F4F4	Surfaces
oooLight	#EBEBEB	OOO shell
oooAccent	#E95420	OOO accent

Fonts: **Inter** (sans), **JetBrains Mono**. Logo: storefront/public/xyz-london-logo.png (masked so it follows currentColor).

Physical product metadata keys

These are stored on the Medusa product and rendered by storefront/src/lib/util/physical-product-copy.ts :

display_title , tagline , tagline_{color} , taglines , collection_line , overview , type_label , origin_label , fit_label , composition , fabric_weight , design_details , core_product_id , item_number , fit , care , size_info , size_info_detail , shipping_copy , returns_copy , size_guide_title , size_guide_intro , size_guide , swatch_colors .

Seeded / scripted XYZ products

Handle	Line	Type
x-money-orders-tee	X	Tee (Black / Green / Purple × S-XL)
x-monogram-tonal-stealth-cap	X	Cap
x-monogram-flat-peak-snapback-cap	X	Cap
y-vintage-patina-luxe-tee	Y	Tee
y-26-suede-mesh-trucker-cap	Y	Cap
z-script-logo-luxe-tee	Z	Tee
z-26-vintage-washed-organic-cap	Z	Cap

Images for scripts live in backend/src/scripts/assets/ .

Boilerplate seed.ts also creates demo Medusa T-Shirt / Sweatshirt / etc. Those can be deleted in Admin or via clear-products.ts (destructive — see §21).

14. Virtual try-on

Provider: **FASHN** (https://api.fashn.ai/v1), overridable with VIRTUAL_TRYON_PROVIDER_BASE_URL .

Enabled when VIRTUAL_TRYON_API_KEY (or FASHN_API_KEY) is set on the storefront.

Route	Role
GET /api/try-on/enabled	Whether try-on is configured
POST /api/try-on/start	Start a job. Body: digital { slug, modelImage } or physical { productHandle, countryCode, modelImage }

Route	Role
GET /api/try-on/status	Poll provider
GET /api/try-on/preview	HMAC-signed preview of a stored file

modelImage must be a data:image/... URL or https:// URL.

Completed outputs are written to MinIO:

- Digital: digital-tryon/{slug}/{predictionId}.png
- Physical: physical-tryon/{country}/{handle}/{predictionId}.png

The marketing page /virtual-try-on is Sanity-driven (arFitPage). Try-on UI also appears on physical and digital PDPs when the API key is present.

15. Digital Form commerce

1. Editors add products on Sanity **Digital Form Page** (digitalProducts[]).
2. Fields: slug, name, line, category, description, images, price (major units, e.g. 45 = £45), currency, platforms, optional medusaHandle, external marketplace URLs, shape, featured / coming soon.
3. If medusaHandle is set, the storefront can enrich with live Medusa pricing.
4. Purchase path does **not** use the Medusa cart. It requires a completed try-on file, then Stripe Checkout.
5. Success page: /{country}/digital/{slug}/success?session_id=cs_...
6. File delivery: /api/digital/download?session_id=cs_... (redirects to a 10-minute signed URL).

Webhook POST /api/stripe/webhook only **verifies** the event. Fulfilment is on-demand from the Checkout Session id.

16. Email

When emails send

Event	Template	Subscriber
order.placed	order-placed	backend/src/subscribers/order-placed.ts
invite.created / invite.resent	invite-user	backend/src/subscribers/invite-created.ts

Templates: backend/src/modules/email-notifications/templates/. Preview: cd backend && npm email:dev → <http://localhost:3002>.

Follow-up for the client

- Order email replyTo is still info@example.com. Change this in order-placed.ts.
- Invite email still uses Medusa branding. Rebrand invite-user.tsx / base.tsx.
- Verify the Resend/SendGrid domain before production orders.

Digital Stripe Checkout uses **Stripe's** receipts, not these templates.

17. Search

1. Backend: set `MEILISEARCH_HOST` + `MEILISEARCH_ADMIN_KEY` so the plugin indexes products (`title` , `description` , `handle` , `variant_sku` , `thumbnail`).
2. Storefront: set `NEXT_PUBLIC_SEARCH_ENDPOINT` , `NEXT_PUBLIC_SEARCH_API_KEY` (**search-only** key), `NEXT_PUBLIC_INDEX_NAME` .
3. Set `NEXT_PUBLIC_FEATURE_SEARCH_ENABLED=true` to show Search in the nav.

UI: `/search` (InstantSearch modal) and `/results/[query]` . Algolia leftovers exist in comments / unused `algoliasearch` dependency; MeiliSearch is what is wired.

`store.config.json` from the Medusa starter is **not** used.

18. File storage (MinIO)

When MinIO env vars are set, the backend uses `backend/src/modules/minio-file/` :

- Auto-creates bucket (default `medusa-media`)
- Sets a **public-read** bucket policy
- Object keys are ULIDs
- Public URLs are stored on file records and used as product image URLs

Without MinIO, files go to `backend/static` and are served at `{BACKEND_URL}/static` .

The storefront uses the same bucket (different key prefixes) for try-on persistence via the AWS S3 SDK (`forcePathStyle: true`).

If product images 404 in production: check bucket policy, `NEXT_PUBLIC_MINIO_ENDPOINT` in `next.config.js` `remotePatterns`, and CORS on MinIO.

19. SEO

Piece	Source
Default title / description	<code>storefront/src/lib/seo/site.ts</code> — “XYZ London Premium Luxury Streetwear & Refined Headwear”
Per-page overrides	<code>Sanity</code> (<code>seoTitle</code> / <code>seoDescription</code> on each page type + <code>staticPageSeo</code>)
Organization JSON-LD	Root layout + <code>siteSettings</code>
Product JSON-LD	Physical PDP (<code>buildProductSchema</code>)
Sitemap	<code>storefront/src/app/sitemap.ts</code> — static core paths × <code>NEXT_PUBLIC_DEFAULT_REGION</code> only (not every country, not every product)
Robots	<code>storefront/src/app/robots.ts</code> — allow <code>/</code> , disallow <code>/api/</code>
Manifest / icons	<code>manifest.ts</code> , <code>icon.svg</code> , <code>apple-icon.svg</code>
Open Graph locale	<code>en_GB</code>

Canonical / OG URLs use `NEXT_PUBLIC_BASE_URL` . Wrong value = wrong canonicals. Set it to the real https domain.

`siteSettings.heroHeadline` / `heroSubheadline` are **legacy** — the storefront reads hero copy from `homePage` .

20. Custom backend APIs and admin widgets

HTTP routes (`backend/src/api/`)

Method	Path	Auth	Purpose
GET	<code>/admin/custom</code>	Admin	Smoke test, empty 200
GET	<code>/store/custom</code>	Public	Smoke test, empty 200
GET	<code>/key-exchange</code>	Public	Returns <code>{ publishableApiKey }</code> for the key titled <code>Webshop</code>

`/key-exchange` exists for Railway auto-config. Restrict or remove it if the backend URL is public and you do not want the publishable key enumerable. A publishable key is not a secret, but there is no reason to advertise it.

No custom `middlewares.ts` . No custom workflows or cron jobs (only README stubs).

Admin widgets (`backend/src/admin/widgets/`)

File	Zone	Purpose
<code>product-storefront-copy.tsx</code>	<code>product.details.after</code>	Storefront metadata editor
<code>product-storefront-swatches.tsx</code>	<code>product.details.after</code>	Color → hex map
<code>product-storefront-collection.tsx</code>	<code>product.details.side.after</code>	Assign X/Y/Z collection
<code>product-storefront-categories.tsx</code>	<code>product.details.side.after</code>	Assign / create type categories
<code>category-storefront-settings.tsx</code>	<code>product_category.details.after</code>	coming soon, hide, shape, subtitle

Widgets call standard Admin REST with `credentials: "include"` . They do not add custom APIs.

21. Catalog scripts

Run from `backend/` with the backend env loaded:

```
pnpm exec medusa exec ./src/scripts/<file>.ts
```

Script	Purpose
<code>seed.ts</code>	Boilerplate seed: EUR store, Europe region (<code>gb de dk se fr es it</code>), warehouse in Copenhagen, shipping €10, publishable key “Webshop”, demo products
<code>ensure-xyz-collections.ts</code>	Create collections X/Y/Z; hide X/Y/Z categories ; normalize Tees handle; assign money-orders tee to X
<code>create-x-money-orders-tee.ts</code>	Full X tee + uploads

Script	Purpose
<code>add-x-money-orders-tee-green.ts</code> / <code>...-purple.ts</code>	Extra colors
<code>update-x-money-orders-tee-media.ts</code>	Refresh images
<code>create-y-vintage-patina-luxe-tee.ts</code>	Y tee
<code>create-z-script-logo-luxe-tee.ts</code>	Z tee
<code>create-x-monogram-caps.ts</code>	Two X caps
<code>create-y-26-suede-mesh-trucker-cap.ts</code>	Y cap
<code>create-z-26-vintage-washed-organic-cap.ts</code>	Z cap
<code>sync-storefront-copy.ts</code>	Batch metadata (fit labels, size guides)
<code>list-products.ts</code>	Log handles
<code>list-admin-users.ts</code>	Log admin emails
<code>reset-admin-password.ts</code>	Requires <code>MEDUSA_ADMIN_EMAIL</code> + <code>MEDUSA_ADMIN_PASSWORD</code>
<code>clear-products.ts</code>	Deletes all products, collections, reservations

Do **not** run `clear-products.ts` on production unless the client explicitly wants a catalog wipe.

`pnpm seed` / first `pnpm ib` will re-create demo Medusa products if the DB is treated as empty. Production databases that already have data are not re-seeded by `init-backend`.

22. Testing

Playwright E2E: `storefront/e2e/`. See `storefront/e2e/README.md`.

- Requires a Postgres database whose name starts with `test_`.
- Database is dropped/recreated between runs.
- Specs cover cart, checkout, search, login, register, discounts, gift cards, profile, addresses, orders.
- Tests assume **Medusa starter seed products** (e.g. “Sweatshirt”), not XYZ handles or Sanity pages.
- There is **no** E2E coverage for Digital Form, OOO, Journal, or try-on.

```
cd storefront
pnpm test-e2e
```

23. Known issues and follow-up work

These are honest leftover items for the incoming developer — not blockers for content editing.

1. **Order email** still uses `replyTo: info@example.com` and generic copy.

2. **Invite email** still says Medusa.
 3. **Mobile side menu** copyright still says “Medusa Store” in places.
 4. **STRIPE_WEBHOOK_SECRET** is not passed into the Medusa Stripe provider (`medusa-config.js` comment).
 5. `/key-exchange` is public.
 6. **No backend** `.env.template` in git.
 7. **Sanity** `homePage.arFitCtaLabel` is read/seeded but **missing from the Studio schema** — editors cannot change the homepage try-on CTA in CMS.
 8. `siteSettings` **hero fields** are unused; hero lives on `homePage` .
 9. **No Sanity live preview / draft mode.**
 10. **No singleton desk** — editors can accidentally create a second Home Page.
 11. **Sitemap** lists only the default region and static paths — not products, journal slugs, or all countries.
 12. **Default region env** is `us` , but seed creates a Europe region without the US. Set `NEXT_PUBLIC_DEFAULT_REGION=gb` (or add a US region).
 13. **Build ignores ESLint and TypeScript errors.**
 14. **E2E tests** do not match the XYZ catalog.
 15. `@medusajs/draft-order` is a dependency but not registered in `medusa-config.js` .
 16. **PayPal** UI exists; backend payment module does not register PayPal.
 17. Root **README** is still the generic Railway boilerplate (version 2.12.1, no XYZ IA). Prefer this handover file.
 18. **Legal body copy** requires a code change / deploy — not CMS.
-

24. Security notes

- Publishable Medusa keys and Stripe **publishable** keys are public by design. Never expose `JWT_SECRET` , `COOKIE_SECRET` , Stripe **secret** keys, Resend keys, MinIO secrets, MeiliSearch **admin** keys, `VIRTUAL_TRYON_API_KEY` , or `SANITY_TOKEN` with write access.
 - Storefront try-on and digital checkout keys are **server-only**. Do not prefix them with `NEXT_PUBLIC_` .
 - Digital download checks Stripe `payment_status` before signing a 10-minute MinIO URL.
 - Preview URLs are HMAC-signed (`DIGITAL_PREVIEW_HMAC_SECRET`).
 - Admin is cookie/JWT authenticated. Use strong `JWT_SECRET` / `COOKIE_SECRET` in production and HTTPS.
 - Set CORS explicitly in production. Do not leave store CORS open to `*` .
 - Rotate all secrets as part of handover (Railway, Stripe, MinIO, Sanity tokens, admin passwords).
 - `medusa-config.js` **logs the full config JSON on startup**, including the fact that modules loaded. Avoid logging secrets; review Railway logs if you add more `console.log` .
-

25. Common tasks

Change homepage headline

Sanity → Home Page → Hero → Publish.

Add a Journal article

Sanity → Journal Post → fill title, slug, category, body, `publishedAt` → Publish. Appears on `/journal` automatically.

Add a Digital product

Sanity → Digital Form Page → add a `digitalProducts` row (slug without `/digital/`) → upload images → Publish.

Hide a Physical Form category without deleting it

Admin → Category → enable **Hide on store**.

Mark a category as Future Forms / Coming Soon

Admin → Category → enable **Coming soon**, optionally set **shape**.

Change shipping price

Admin → Settings → Locations & shipping → edit the shipping option. Seed created Standard + Express at €10 / \$10.

Add a country

Admin → Settings → Regions → add the country to Europe (or create a new region + currency + tax + providers). Storefront URLs become `{iso}/...` after middleware cache refreshes (up to 1 hour, or restart).

Point the store at a new domain

1. DNS → Railway (or new host).
2. Storefront `NEXT_PUBLIC_BASE_URL=https://your-domain`.
3. Backend `BACKEND_PUBLIC_URL`, `STORE_CORS`, `ADMIN_CORS`, `AUTH_CORS`.
4. Stripe webhook URLs.
5. Sanity image / site settings if they contain absolute URLs.

Rebuild after env change (backend)

```
rm -rf backend/.medusa/server
cd backend && pnpm build && pnpm start
```

Preview emails

```
cd backend && pnpm email:dev
```

26. Official documentation

Use these for platform questions; use this handover for **this** project's customisations.

- Medusa 2: <https://docs.medusajs.com>
- Medusa Admin: `{BACKEND}/app` + <https://docs.medusajs.com/user-guide>
- Medusa Stripe: <https://docs.medusajs.com/resources/commerce-modules/payment/payment-provider#stripe>
- Next.js 15: <https://nextjs.org/docs>
- Sanity: <https://www.sanity.io/docs>
- MeiliSearch plugin: <https://github.com/rokmohar/medusa-plugin-meilisearch>
- FASHN try-on API: <https://docs.fashn.ai> (provider default)

- Railway Medusa template background: root `README.md` and <https://funkyton.com/medusajs-2-0-is-finally-here/>

27. Glossary

Term	Meaning
Physical Form	Tangible catalog (<code>/store</code>) powered by Medusa
Digital Form	Digital / NFT-style merchandising in Sanity + Stripe Checkout
OOO	“Highest Expression” / private list — route <code>/private-expressions</code>
Line (X / Y / Z)	Design line = Medusa collection <code>x / y / z</code>
Type category	Tees, Caps, etc. — Medusa product category
Future Forms	Coming-soon type categories
Publishable key	Public Medusa Store API key (header <code>x-publishable-api-key</code>)
Handle	URL slug for a Medusa product or category
Region	Medusa market (currency + countries). Drives <code>/ {country} /</code>
Singleton	Sanity document that should exist exactly once (Home, About, ...)
GROQ	Sanity query language used by the storefront
FASHN	Third-party virtual try-on API
init-backend	Railway helper that migrates/seeds on empty DBs

Handover sign-off checklist

Copy this into your transfer email and tick with the client.

- ☐ Git repository transferred or forked into the client org
- ☐ Railway (or host) project transferred; all env vars documented offline
- ☐ Client Medusa Admin user created; old passwords rotated
- ☐ Sanity project `bff91fb2` — client is Owner/Admin
- ☐ Stripe account + webhooks pointed at production URLs
- ☐ Resend/SendGrid domain verified; `info@example.com` replaced
- ☐ MinIO bucket public-read; images load on the live storefront
- ☐ `NEXT_PUBLIC_DEFAULT_REGION` matches a real Medusa country
- ☐ CORS set to the live storefront origin
- ☐ Test physical checkout (Stripe test, then live)
- ☐ Test digital checkout + download if Digital Form is in scope
- ☐ OOO HubSpot (or override) form works
- ☐ DNS / SSL for storefront and backend
- ☐ This `HANDOVER.md` stored with the repo

Generated from the XYZ London monorepo as of September 2026. Medusa 2.15.2, Next.js 15, Sanity 3. Update this file when you add modules, env vars, or routes.

This PDF is generated from HANDOVER.md. Prefer the markdown file in the repository when copying commands. Do not commit secrets into either document.